
StepCAD: Mesh-to-CAD Code Generation via LLM Policy and Geometry-Guided Search

Ghadi Nehme Faez Ahmed
Department of Mechanical Engineering
Massachusetts Institute of Technology
Cambridge, MA 02139, USA
{ghadi, faez}@mit.edu

Abstract

Recovering executable CAD programs from 3D meshes is fundamentally challenging due to the long-horizon, compositional nature of CAD construction and the need for precise estimation of both discrete operations and continuous parameters. Existing learning-based methods primarily treat mesh-to-CAD reconstruction as one-shot sequence prediction and are largely limited to simple sketch-extrude pipelines, restricting operation diversity and preventing the use of intermediate geometric feedback, with no mechanism for refining generated programs. To address this limitation, we introduce **StepCAD**, a generative optimization approach that performs step-by-step, geometry-conditioned program synthesis followed by explicit refinement in program space. Given an input mesh, StepCAD first predicts construction sequences using a state-conditioned CAD policy conditioned on both target and intermediate geometry, then refines them through IoU-guided tree search over local program edits. To address the limited operation diversity in prior work, we further introduce **ARCADE-1.5M**, a large-scale dataset of 1.5M executable CAD programs with diverse operations, long-horizon sequences up to 150+ operations, and 12.5M intermediate state-action transitions. Experiments on multiple CAD reconstruction benchmarks show that StepCAD achieves state-of-the-art reconstruction accuracy, validity, and robustness, with up to **87.2%** relative IoU improvement over the best prior method and increasingly larger gains as shape complexity increases.

1 Introduction

Recovering structured, executable CAD programs from 3D meshes is a critical capability for modern engineering workflows. Many industrial pipelines produce dense, unstructured 3D geometry from 3D scans of physical parts [1, 2], topology-optimized designs [3, 4], and increasingly from generative 3D models [5, 6, 7] that must be converted into editable parametric representations to enable design iteration, manufacturing, and downstream simulation. Unlike conventional 3D reconstruction, which approximates surface geometry, *mesh-to-CAD generation* seeks to recover a sequence of parametric operations that reconstructs the shape compactly and faithfully. This task is inherently challenging: CAD programs are long-horizon, compositional, and highly structured, with strong dependencies between operations and continuous parameters that must be estimated with high precision [8, 9].

Recent learning-based approaches model CAD reconstruction as autoregressive program generation conditioned on geometry [8, 2, 10, 11]. While effective at producing plausible sequences, these methods suffer from three fundamental limitations. First, they rely on *restricted operation vocabularies*, often dominated by sketch-extrude pipelines, failing to capture the diversity of real-world

CAD modeling [8, 2, 12]. Second, they treat generation as a *one-shot prediction problem*, without incorporating geometric feedback at each step preventing the model from adapting its decisions based on the evolving shape and often leading to invalid or geometrically inaccurate programs. Third, purely generative models *lack the precision* required to faithfully reproduce target geometry, particularly when continuous parameters such as extrusion depths or sketch dimensions must be estimated accurately.

We argue that mesh-to-CAD reconstruction should be reformulated as policy-driven, geometry-conditioned program synthesis with search-based refinement. This perspective decomposes the problem into two complementary components: a learned policy that conditions on intermediate geometry to propose candidate construction sequences, and a geometry-aware optimization procedure that refines these sequences to improve geometric fidelity. The formulation leverages the strengths of generative models for capturing global structure while compensating for their lack of precision through explicit search guided by geometric objectives—a paradigm with a long history in program synthesis [13, 14, 15] that we adapt to the unique constraints of CAD generation.

Realizing this paradigm requires both a model capable of state-conditioned generation and training data that exposes diverse construction trajectories—neither of which is well-supported by existing CAD datasets. We therefore introduce two contributions in tandem. First, ARCADE-1.5M, a large-scale synthetic dataset of 1.5M executable CAD programs spanning extrude, revolve, loft, sweep, Boolean operations, and finishing features, with sequences of up to 150 steps and 12.5M intermediate state-action transitions providing dense supervision over construction trajectories. Second, StepCAD, a generative optimization [16] approach for mesh-to-CAD reconstruction that, given an input shape, generates a candidate program using a state-conditioned LLM policy trained on ARCADE-1.5M, then refines it through a geometry-aware tree search over local edits—combining continuous parameter optimization with discrete structural modifications, all guided by IoU.

This combination of policy and optimization yields substantial improvements in reconstruction accuracy and robustness. We show that precise numerical estimation plays a critical role in CAD generation, and that explicit refinement in program space significantly improves geometric fidelity beyond what is achievable through generation alone. StepCAD achieves state-of-the-art (SOTA) performance on CADBench [17], a CAD benchmark spanning ABC [18], Fusion 360 [19], DeepCAD [8], MCB [20], and Objaverse [21], with increasingly larger gains as shape complexity grows, precisely where prior approaches degrade most severely.

In summary, our contributions are:

- **ARCADE-1.5M: a large-scale CAD program dataset.** We introduce a dataset of 1.5M executable CAD programs with rich operation diversity, long-horizon construction sequences (up to 150 steps), and 12.5M intermediate state-action transitions, substantially expanding the scale and complexity of available CAD data.
- **State-conditioned CAD policy.** We propose a point-cloud-conditioned autoregressive policy that predicts CAD operations given both intermediate and target geometry.
- **Program-space optimization for CAD generation.** We reformulate mesh-to-CAD reconstruction as optimization over program space and introduce a geometry-aware tree search that refines candidate programs through IoU-guided local edits.
- **State-of-the-art performance on diverse CAD benchmarks.** We achieve SOTA results across five diverse benchmarks, with up to **87.2%** relative improvement in IoU over the best prior method, demonstrating strong robustness and growing advantages as shapes become more complex.

2 Related Work

Learning-Based CAD Program Generation. While text-to-CAD generation has attracted significant attention [22, 23, 24], industrial workflows more often require recovering CAD programs from existing 3D geometry—such as scanned parts [1, 2, 25], topology-optimized geometry [3, 4, 26], or AI-generated meshes [5, 6, 7]—where the design already exists in shape but lacks editable parametric structure. Recent work has therefore explored neural approaches for generating CAD programs as sequences of parametric operations from 3D shapes [2, 9, 10], motivated by applications in reverse engineering, scalable dataset construction, and bridging geometry-centric 3D generation [5, 6] with

editable, parametric workflows. Early methods such as DeepCAD [8] introduce domain-specific languages for sketch-and-extrude programs and learn sequence autoencoders over this representation, with subsequent works extending the paradigm to richer inputs and structured decoders [27, 28, 9]. More recently, large language models have been applied to CAD program generation from images [7, 29, 30] and point clouds [2, 10, 11], enabling flexible conditioning across modalities. Despite these advances, existing methods rely primarily on amortized autoregressive prediction, which often leads to limited geometric precision due to its single-shot nature. In contrast, StepCAD explicitly separates program generation from program optimization, using a learned policy to propose structurally consistent programs and a search procedure to refine them for geometric accuracy.

Iterative Refinement and Neural-Guided Program Search. A small but growing line of work explores iterative refinement for CAD generation. CADReasoner [31] introduces LLM-based refinement guided by visual feedback, CADCrafter [29] applies direct preference optimization with a code-validity checker, and CADFusion [23] alternates supervised and visual-feedback stages during training. While these methods incorporate feedback, refinement remains coupled to the generation stage and operates implicitly through training-time supervision. More broadly, combining learned policies with explicit search has a long history in program synthesis [13, 14, 15] and recent code generation [32, 33, 34], where neural networks propose candidates that are refined by search to satisfy correctness criteria. StepCAD adopts this paradigm in the CAD setting: an LLM policy proposes structurally consistent programs, while a geometry-aware tree search refines them through local edits including continuous parameter optimization, sketch modification, and operation skipping with candidates ranked by IoU against the target. Compared to general code generation, CAD program search is uniquely constrained by both executability and continuous parameter precision, motivating geometry as the primary search signal.

Geometry-Driven CAD Reverse Engineering. Another line of work recovers CAD structure directly from 3D shape. Classical pipelines decompose shapes into primitives and fit analytic surfaces but struggle to produce consistent parametric programs for complex designs. Learning-based approaches improve robustness by combining neural segmentation with surface fitting, particularly for boundary representation (B-rep) reconstruction [25, 35, 36, 37, 1]. While these methods achieve high geometric fidelity, they do not recover ordered construction sequences or parametric design intent. Inverse constructive solid geometry (CSG) methods [38] formulate reconstruction as a combinatorial search over Boolean programs, but are typically restricted to simple primitive sets and do not scale to sketch-based modeling, revolutions, or finishing features. In contrast, StepCAD explicitly models the sequential construction process and produces executable programs, enabling both interpretability and editability.

CAD Datasets. Large-scale datasets such as ABC [18], DeepCAD [8], and Fusion360 [19] have enabled learning structured CAD representations. However, they are largely restricted to sketch-and-extrude operations and short construction sequences. Concurrent efforts such as CAD-Recode [2], Zero-to-CAD [12], and CADEvolve [39] expand operation diversity through procedural and evolutionary synthesis, yet remain limited in sequence length and geometric complexity. Our ARCADE-1.5M dataset is designed to provide long-horizon, diverse, and executable CAD programs with dense intermediate supervision, enabling effective training of state-conditioned policies.

3 Method

We hypothesize that learning CAD programs from geometry requires large-scale procedural data with rich operation diversity, which is lacking in existing datasets. To address this, we introduce **ARCADE (Artificial Repository of CAD Examples)**, a large-scale synthetic dataset of executable CAD programs with diverse operations and workflow-consistent structure, providing dense supervision over construction trajectories. We then propose **StepCAD**, a generative optimization approach for mesh-to-CAD reconstruction: a **state-conditioned LLM policy** generates candidate programs, followed by **geometry-aware optimization** in program space. This decomposition enables learning global construction structure while achieving high geometric fidelity.

3.1 Problem Statement

We study **mesh-to-CAD reconstruction**: recovering an executable parametric CAD program that reproduces a target 3D shape. Let $\mathcal{M}$ denote the space of meshes and $\mathcal{P}$ the space of valid

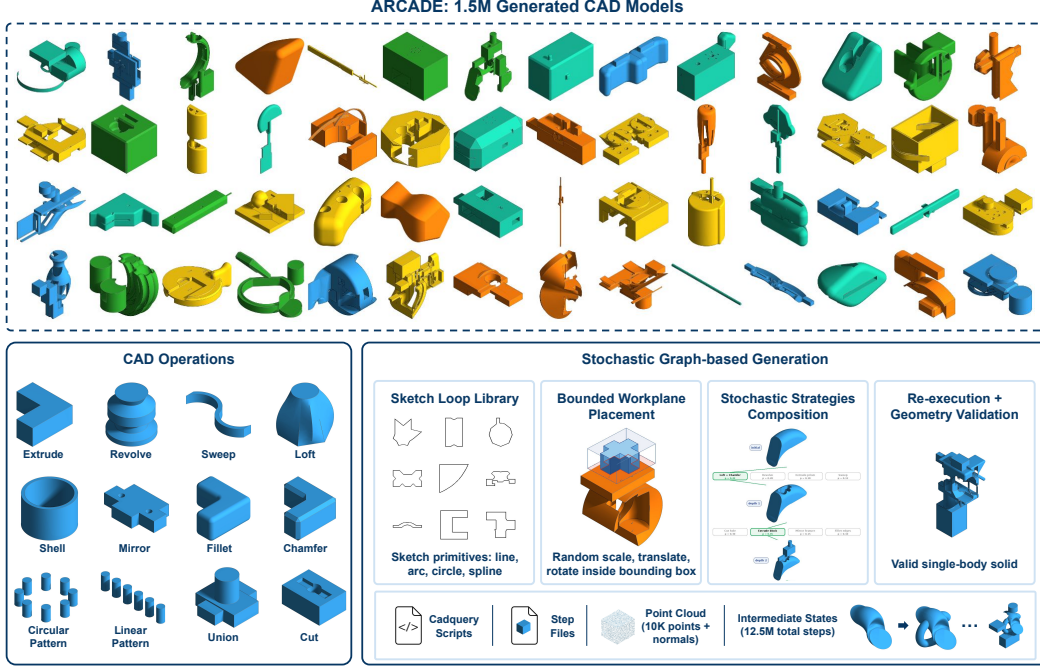


Figure 1: **ARCADE-1.5M: large-scale synthetic CAD program generation.** Our pipeline synthesizes diverse long-horizon CAD programs through stochastic strategy-guided composition of sketch primitives and CAD operations. Each program is executed and validated, yielding 1.5M executable programs and 12.5M intermediate construction states with broad operation diversity.

CAD programs. A program $p = (a_1, \dots, a_T) \in \mathcal{P}$ is an ordered sequence of parametric operations whose execution produces geometry $S(p) \in \mathcal{M}$. Given an input mesh $m \in \mathcal{M}$, the goal is to predict a valid program $\hat{p}$ such that the generated geometry matches the target shape: $\hat{p} = \arg \max_{p \in \mathcal{P}} \text{IoU}(S(p), m)$. The problem is challenging due to the combinatorial structure of $\mathcal{P}$, long-horizon autoregressive dependencies, and the existence of multiple valid programs producing similar geometry. We represent CAD programs using CadQuery code [40].

3.2 ARCADE-1.5M: Constraint-Guided Synthetic CAD Programs

We introduce **ARCADE-1.5M**, a dataset of **1.5M** executable parametric CAD programs paired with valid 3D geometry. ARCADE-1.5M combines stochastic geometry generation with workflow-inspired constraints to synthesize long-horizon, structurally plausible CAD construction sequences at scale (Fig. 1). Dataset statistics and generation details can be found in Appendices F and G.

Constraint-guided primitive generation. We initialize generation from a library of 2D sketch loops extracted from ABC, including lines, arcs, circles, and splines. Sketches are randomly perturbed, rotated, and scaled, then placed on randomly selected faces of a target bounding box, providing explicit control over feature size, placement, and orientation. Each sketch is converted into a solid using randomly sampled **extrude**, **revolve**, **loft**, or **sweep** operations with stochastic parameters. Optional fillets and chamfers are applied probabilistically to increase geometric diversity.

Workflow-inspired composition constraints. Rather than composing disconnected primitives, ARCADE-1.5M enforces constraints inspired by practical CAD workflows. All intermediate states must remain valid single-body solids with positive volume. Additive operations require controlled overlap with the host body, while subtractive operations are accepted only under sufficient volumetric intersection. These constraints prevent degenerate geometry and preserve physically meaningful feature placement.

Strategy-based hierarchical construction. To generate long, coherent programs, ARCADE-1.5M uses **94 heuristic generation strategies** that constrain operation types and approximate sequence

Dataset	Size	Has Construction Sequence	Avg Ops / Seq	Avg Solid Ops / Seq	Avg Faces	Solid Operation Coverage (%)							
						Extrude	Revolve	Sweep	Loft	Fillet	Chamfer	Shell	Mirror
DeepCAD [8]	178k	✓	15	1.5	10	100	0	0	0	0	0	0	0
ABC-mono [41]	200k	✓	*	*	*	100	0	0	0	0	0	0	0
KOCAD [41]	4.6k	✓	*	*	*	100	0	0	0	0	0	0	0
Fusion 360 [19]	8.6k	✓	18	2.2	16	100	0	0	0	0	0	0	0
CAD-Recode [2]	1M	✓	28	2.0	17	100	0	0	0	0	0	0	0
Zero-to-CAD [12]	1M	✓	24	5.5	44	85.99	18.73	3.88	4.17	36.92	74.66	15.00	1.56
CADEvolve [39]	1.3M	✓	28	5.5	54	83.05	4.80	5.75	8.48	27.78	4.76	1.95	0.08
ABC [18]	1M	✗	—	—	114	—	—	—	—	—	—	—	—
ARCADE-1.5M (ours)	1.5M	✓	40	8.5	86	95.42	12.13	25.37	13.10	34.77	33.66	10.01	6.70

Table 1: **Comparison of CAD datasets with operation coverage (%)**. Operation coverage is the percentage of programs containing at least one instance of the operation. Prior datasets are mostly sketch-extrude only (100% extrude). Concurrent works CADEvolve and Zero-to-CAD add parametric diversity, but ARCADE-1.5M is largest (1.5M), longest (40 ops/seq, 8.5 solid), and most geometrically complex (86 faces), with the broadest coverage of sweep, loft, and mirror operations. *: indicates datasets that are not publicly available.

lengths. Strategies emulate common modeling patterns, including additive construction, cut-heavy machining, shelling, mirroring, lofting, sweeping, and finishing operations. This hierarchical structure enables realistic long-horizon programs while avoiding combinatorial explosion. Generated sequences range from 3 to 150+ operations (mean: 40.3, median: 33).

Validation and normalization. Every sample undergoes double validation: the in-memory CadQuery solid must be valid, and the exported Python program is re-executed in a clean namespace before STEP export. Shapes are normalized to a canonical $[-1, 1]^3$ bounding box, and floating-point literals are rounded to three decimals to reduce token complexity.

Dataset characteristics. Each sample contains a CadQuery script, STEP B-Rep, triangulated mesh, and a 10K-point cloud with normals. We additionally store intermediate geometry after every **solid operation**, yielding approximately **12.5M** construction states for dense trajectory supervision. ARCADE-1.5M covers workplanes, sketch primitives, volumetric and Boolean operations, shells, mirrors, fillets, chamfers, and spatial patterns. Compared to prior datasets, it uniquely combines scale, long-horizon programs, and rich operation diversity, with substantially higher geometric complexity (average face count: 86). Overall, ARCADE-1.5M bridges executable CAD construction logic and resulting 3D geometry, enabling training for realistic mesh-to-CAD generation.

3.3 StepCAD: Generative Optimization for Mesh-to-CAD Reconstruction

We propose **StepCAD**, a two-stage framework that combines a *state-conditioned autoregressive CAD policy* (II-StepCAD) with *geometry-aware program optimization* (Fig. 2). The policy generates a structurally plausible executable program, while a subsequent local search stage refines the program to improve geometric fidelity.

3.3.1 II-StepCAD: State-Conditioned CAD Policy

Sequential generation. Direct optimization in program space is combinatorial and highly non-convex. We instead learn an autoregressive policy

$$\pi_{\theta}(a_t \mid Q_{t-1}, Q_m, a_{<t}),$$

where Q_m is a point cloud sampled from the target mesh, Q_{t-1} is sampled from the intermediate geometry, and $a_{<t}$ denotes previously generated operations. This induces a distribution over executable CAD programs:

$$\pi_{\theta}(p) = \prod_{t=1}^T \pi_{\theta}(a_t \mid Q_{t-1}, Q_m, a_{<t}).$$

Geometry conditioning. Target and intermediate geometries are encoded using a shared Point-MAE encoder [42]:

$$z_m = f_{\phi}(Q_m), \quad z_{t-1} = f_{\phi}(Q_{t-1}),$$

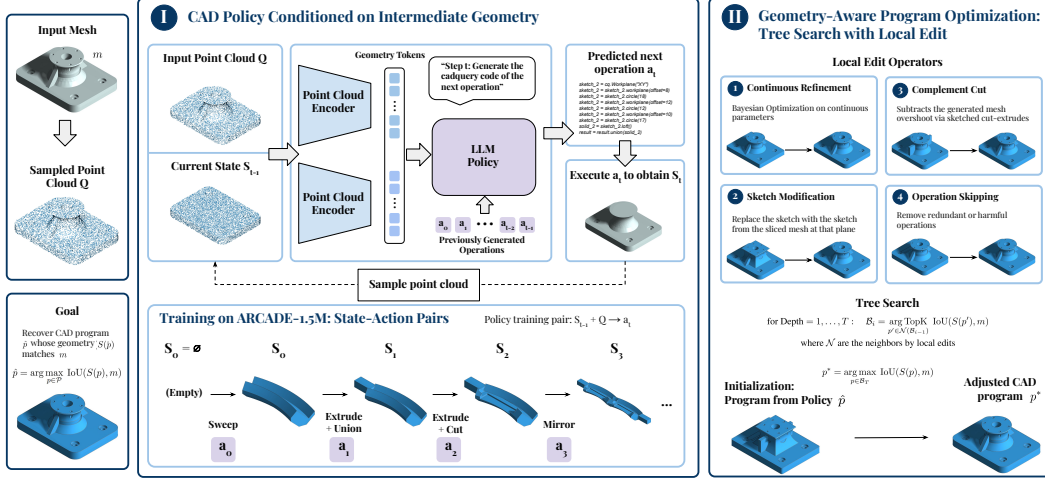


Figure 2: **StepCAD pipeline.** A state-conditioned CAD policy generates an executable program from an input shape, which is subsequently refined via geometry-aware search over local program edits.

and injected as conditioning tokens into the autoregressive model.

Policy architecture. We instantiate π_θ using Qwen2.5-Coder-1.5B [43] conditioned on geometry embeddings and prior program tokens. The model predicts executable CAD operations directly in token space.

Training. The policy is trained with teacher forcing over dense state-action trajectories extracted from ARCADE-1.5M:

$$\mathcal{L}(\theta) = -\mathbb{E} [\log \pi_\theta(a_t | Q_{t-1}, Q_m, a_{<t})].$$

Supervision at every intermediate construction step improves long-horizon stability and geometric consistency.

Closed-loop inference. At inference, generation alternates between policy prediction and CAD execution. Starting from an empty state, the model performs inference-time sampling to predict the next operation, executes the updated CadQuery program, samples the resulting geometry, and re-conditions on the produced intermediate shape before continuing generation. Generation terminates once reconstruction quality plateaus, an end token is emitted, or a maximum length is reached (Details in Appendix H). This produces a seed program $\hat{p}$ that captures the global structure of the target shape and initializes the refinement stage.

3.3.2 Geometry-Aware Program Optimization

Given a seed CAD program $\hat{p} \sim \pi_\theta$, we perform local search in program space:

$$p^* = \arg \max_{p \in \mathcal{N}(\hat{p})} \text{IoU}(S(p), m),$$

where $S(p)$ denotes the executed geometry and $\mathcal{N}(\hat{p})$ is defined by local program edits.

Direct optimization over all edits is exponential in the program length T . We therefore perform a geometry-aware beam search over operations in execution order. At step t , each candidate program in the beam is expanded using a small set of local edits applied to operation a_t , executed in CadQuery, and ranked by IoU against the input mesh: $\mathcal{B}_t = \text{TopK}_K \{p' \in \text{Expand}(p, t) : p \in \mathcal{B}_{t-1}\}$.

We use beam width $K = 2$ and four local edits per expansion, resulting in linear complexity in T while allowing recovery from locally suboptimal decisions. The final refined program is selected as:

$$p^* = \arg \max_{p \in \mathcal{B}_T} \text{IoU}(S(p), m).$$

The local edits are: **Continuous refinement:** optimize continuous operation parameters (e.g., depths, radii, sketch dimensions). **Sketch modification:** replace operation sketches using geometry

extracted from the input mesh. **Complement cut:** remove overshoot regions using automatically generated cut-extrudes. **Operation skipping:** remove redundant or harmful operations.

Implementation Details. All algorithmic and training details are provided in appendices H and I.

4 Experiments, Results, and Discussion

We evaluate **StepCAD** on multiple mesh-to-CAD reconstruction benchmarks, comparing against state-of-the-art methods and analyzing the contributions of each component of our approach. We focus on geometric fidelity, validity of generated programs, and robustness to complex constructions.

Benchmark. We evaluate on **CADBench** [17], a comprehensive benchmark for CAD program generation that aggregates five representative datasets spanning diverse geometric domains and construction complexity. Specifically, CADBench includes: **DeepCAD** [8] and **Fusion 360** [19], **ABC** [18] for large-scale CAD shapes with varying difficulty (Easy, Medium, Hard, based on number of STEP faces [44]), **MCB** [20] for mechanical components, and **Objaverse** [21] for organic shapes. Each benchmark contains 3,000 samples, except ABC which provides 2,000 samples per complexity split. This suite enables robust evaluation across both structured mechanical designs and free-form geometries, providing a holistic assessment of CAD generation performance.

Metrics. We report **Intersection-over-Union (IoU)** measuring volumetric overlap, **Chamfer Distance (CD)** measuring surface discrepancy (lower is better), and **Valid Shape Rate (VSR)**, the percentage of generated programs that execute successfully and yield valid geometry. (Appendix E)

Baselines. We compare against CAD-Recode [2], CADEvolve [39], and Cadrille [10] which are representative learning-based methods for CAD generation from geometry, as well as CADReasoner [31], an iterative CAD refinement for mesh to CAD generation. For all baselines, we use their publicly released checkpoints and evaluate them on CADBench.

Quantitative Results. Table 2 reports IoU and Chamfer Distance on CADBench across structured, mechanical, and organic domains. StepCAD achieves SOTA performance on all benchmarks, with the largest gains on complex shapes.

State-of-the-art mesh-to-CAD reconstruction. StepCAD consistently outperforms prior methods across all datasets. On **ABC Hard**, IoU improves from 0.31–0.39 to **0.73**, an **87.2%** relative gain over the strongest baseline. On **ABC Medium**, IoU increases from 0.43 to **0.74 (72.1% gain)**. On the **Mechanical** and **Organic** subsets, StepCAD reaches **0.89** and **0.71** IoU, improving by **32.8%** and **69.0%** respectively. StepCAD also achieves the lowest Chamfer Distance on every benchmark, with reductions up to **28.0%** on Mechanical, demonstrating robustness across both structured CAD models and free-form geometries.

Benefits of state-conditioned step-by-step generation. Even without optimization, II-StepCAD already surpasses prior work, especially on difficult benchmarks (e.g., 0.52 IoU vs. ~ 0.38 on ABC Hard, $\sim 37\%$ improvement). Unlike one-shot autoregressive approaches, II-StepCAD conditions each prediction on the intermediate geometry produced so far, enabling progressively more accurate structural and parametric decisions throughout construction. We note that CADReasoner performs slightly better than II-StepCAD on the easiest benchmarks (DeepCAD and ABC Easy), likely due to specialization to simpler in-distribution samples. However, its performance degrades substantially on harder subsets, whereas II-StepCAD remains robust. This suggests that explicit intermediate-geometry conditioning generalizes better to complex shapes than iterative learned refinement alone.

Generative optimization is critical. The full StepCAD system substantially improves over the policy alone (e.g., $0.52 \rightarrow 0.73$ IoU on ABC Hard). The geometry-aware tree search corrects both structural errors (e.g., missing features) and parametric inaccuracies (e.g., extrusion depths, sketch dimensions), producing an average IoU gain of 0.14 ($\sim 20\%$ relative). These results show that combining learned proposals with targeted program-space search is essential for high-fidelity CAD reconstruction.

Surface accuracy. Chamfer Distance follows the same trend. StepCAD achieves the lowest error on all datasets (e.g., 0.0263 vs. 0.0283 on ABC Hard, and 0.0420 vs. 0.0446 on Organic), indicating improvements not only in volumetric overlap but also in fine geometric detail.

Table 2: **Mean mesh-to-CAD generation metrics.** IoU and VSR are higher better; CD is lower better. Best overall in bold; second best underlined.

Model	DeepCAD			Fusion 360			ABC Easy			ABC Medium			ABC Hard			Mechanical			Organic		
	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$	IoU $\uparrow$	CD $\downarrow$	VSR $\uparrow$
CAD-Recode	0.79	0.0207	96.22	0.65	0.0268	92.63	0.76	0.0237	95.02	0.37	0.0349	88.35	0.35	0.0328	87.57	0.66	0.0297	94.01	0.40	0.0570	87.84
Cadrille	0.78	0.0211	96.97	0.66	0.0268	94.87	0.75	0.0234	96.58	0.40	0.0359	92.30	0.37	0.0337	92.18	0.65	0.0295	96.80	0.42	0.0549	91.03
CAD-Evolve	0.74	0.0389	99.70	0.63	0.0466	97.60	0.72	0.0413	98.88	0.39	0.0638	95.82	0.39	0.0617	95.27	0.64	0.0486	96.90	0.40	0.0890	96.91
CAD-Reasoner	<u>0.86</u>	<u>0.0197</u>	99.47	0.71	0.0326	98.33	<u>0.83</u>	0.0242	99.60	0.43	0.0419	97.10	0.31	0.0627	92.80	0.67	0.0413	98.80	0.31	0.0968	96.60
Π -StepCAD (Ours)	0.82	0.0205	100.00	<u>0.75</u>	<u>0.0240</u>	99.97	0.77	<u>0.0227</u>	100.00	<u>0.53</u>	<u>0.0320</u>	100.00	<u>0.52</u>	<u>0.0283</u>	99.95	0.78	<u>0.0259</u>	100.00	<u>0.53</u>	<u>0.0446</u>	100.00
StepCAD (Ours)	0.89	0.0191	100.00	0.83	0.0228	99.97	0.89	0.0200	100.00	0.74	0.0302	100.00	0.73	0.0263	99.95	0.89	0.0213	100.00	0.71	0.0420	100.00
Improvement vs. Best Prior (%)	3.5	3.0	0.3	16.9	14.9	1.7	7.2	14.5	0.4	72.1	13.5	3.0	87.2	19.8	4.9	32.8	27.8	1.2	69.0	23.5	3.2

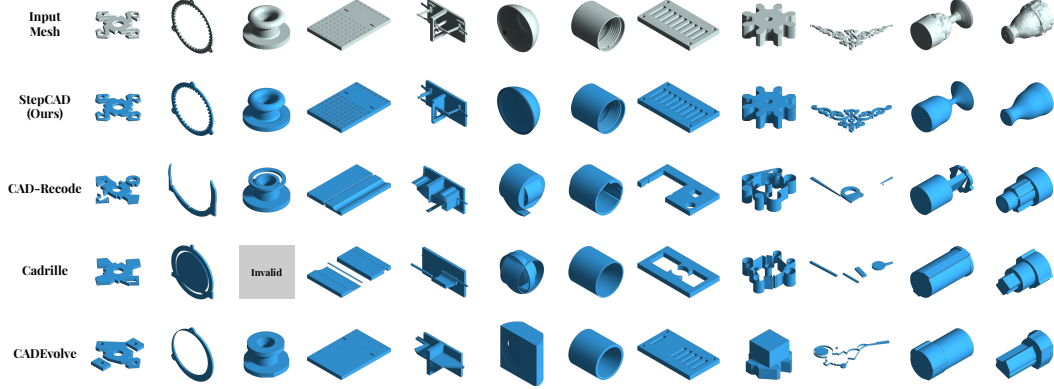


Figure 3: **Qualitative mesh-to-CAD reconstruction results.** Rows show input mesh, StepCAD, CAD-Recode, Cadrille, and CADEvolve; Invalid denotes a failed/non-executable reconstruction.

Disentangling dataset, policy, and optimization. Table 3 isolates the contribution of each component. Training on the more diverse **ARCADE-1.5M** dataset alone already improves over CADEvolve (0.568 $\rightarrow$ 0.632 IoU) even without previous-geometry conditioning or optimization. Since this variant predicts operations directly from the target geometry without conditioning on intermediate states, it behaves similarly to a one-shot autoregressive CAD generator. This highlights the importance of large-scale operation diversity and long-horizon procedural supervision alone. Adding **previous-geometry conditioning** further improves IoU from 0.632 to 0.682 while reducing CD from 0.045 to 0.028. Conditioning on intermediate geometry enables step-by-step generation, allowing the policy to progressively correct structural errors and refine predictions throughout construction. Finally, adding the **geometry-aware optimization** stage yields the largest gain, increasing IoU from 0.682 to 0.816. Together, these results show that high-fidelity mesh-to-CAD reconstruction requires: (i) diverse procedural supervision, (ii) state-conditioned sequential generation, and (iii) targeted program-space optimization.

Table 3: **Ablation of dataset diversity, state-conditioned generation, and geometry-aware optimization.** Training on ARCADE-1.5M improves over CADEvolve dataset, previous-geometry conditioning further improves step-by-step generation, and the tree search yields the largest gain.

Variant	CD $\downarrow$	IoU $\uparrow$	VSR (%) $\uparrow$
CADEvolve	0.056	0.568	97
Π -StepCAD w/o previous-geometry tokens	0.045	0.632	100
Π -StepCAD	0.028	0.682	100
StepCAD	0.025	0.816	100

Qualitative results. Figure 3 shows reconstructions across datasets. StepCAD produces CAD programs that closely match target geometry while preserving both global structure and fine details.

Impact of ARCADE-1.5M and operation diversity. A key factor behind these gains is the broader operation coverage of **ARCADE-1.5M**, including fillets, chamfers, lofts, and revolves. In contrast,

prior datasets are dominated by sketch-extrude operations, biasing models toward simple extrusion-based programs. As a result, StepCAD generates more diverse operations. On the **Organic** subset, lofts appear in 30.4% of generated programs versus 3.0% for CADEvolve, while revolves appear in 23.2% versus 0.8%. Prior methods rely overwhelmingly on extrusions. (Appendix D)

This diversity is critical for reconstructing geometries that cannot be represented well with sketch-extrude primitives alone. For example, in the last column of Fig. 3, StepCAD uses loft operations to recover smooth non-planar transitions, achieving higher fidelity (**IoU**: 0.83, **CD**: 0.066) than CADEvolve (**IoU**: 0.66, **CD**: 0.108) and Cadrille (**IoU**: 0.67, **CD**: 0.101), which primarily rely on extrusion-based constructions. These results demonstrate that operation diversity is essential for faithful reconstruction of complex geometry.

Applications and one-to-many CAD generation. Beyond reconstruction, StepCAD enables several downstream applications enabled by its state-conditioned generative formulation. As shown in Appendix B and Fig. 4, II-StepCAD can generate diverse editable CAD variants from a single reference mesh while preserving the overall structure, enabling rapid design exploration and parametric shape variation. StepCAD can also be integrated into image-to-CAD pipelines (Appendix B, Fig. 5) by reconstructing parametric CAD programs from meshes generated by image-to-3D models such as Hunyuan3D [6]. Finally, Appendix C and Fig. 6 highlight the inherently one-to-many nature of mesh-to-CAD reconstruction: the policy can generate multiple valid CAD programs with different operation sequences that produce nearly identical geometry. This suggests that StepCAD learns a distribution over plausible CAD construction strategies rather than memorizing a single deterministic reconstruction process.

Overall, these results demonstrate the effectiveness of combining state-conditioned generative policies with geometry-aware optimization for robust and accurate CAD program synthesis.

Additional ablations. Additional ablations are provided in the appendix, including the effects of Qwen model scale and point-cloud encoder choice (Appendix A.2, Table 5); policy inputs, PointMAE fine-tuning, and point sampling strategy (Appendix A.1, Table 4); and the contribution of each local edit within the tree-search optimization procedure (Appendix A.3, Table 6).

5 Limitations and Future Work

While **StepCAD** demonstrates strong performance and enables high-fidelity reconstruction of parametric CAD programs, it has several limitations. **Computation and scalability.** The optimization stage increases inference time to ~ 1 minute per sample (vs. ~ 10 seconds for prior methods), with cost growing for long programs. **Heuristic search.** Refinement relies on a fixed set of local edits, which may not fully capture the space of valid CAD transformations and can limit recovery from poor initial proposals. **Synthetic data and ambiguity.** Training on ARCADE-1.5M introduces a domain gap, and mesh-to-CAD reconstruction is inherently ill-posed; optimizing for IoU does not guarantee recovery of design intent or the original construction sequence. **Constraint modeling.** The method does not explicitly model parametric or relational constraints, which are central to CAD editability. **Societal impact.** While enabling faster design and reverse engineering, the method may also facilitate misuse (e.g., replication of proprietary designs) and shift designer roles toward oversight. To address these gaps, future work will: (1) improve efficiency through learned value functions, surrogate metrics, and parallelized or amortized search; (2) learn richer edit operators and adaptive search strategies; (3) incorporate real-world CAD data and develop intent-aware objectives; (4) extend the framework to model parametric and relational constraints; and (5) develop safeguards, attribution mechanisms, and human-in-the-loop systems for responsible deployment.

6 Conclusion

We presented **StepCAD**, a generative optimization approach for mesh-to-CAD reconstruction that combines a state-conditioned CAD policy with geometry-aware program refinement. By reformulating CAD generation as optimization in program space, our approach overcomes the limitations of one-shot sequence prediction and achieves SOTA performance across multiple benchmarks. We further introduced **ARCADE-1.5M**, a large-scale dataset of CAD programs with diverse operations and long-horizon construction sequences, enabling effective training of state-conditioned models. Our results demonstrate that combining learned generation with explicit geometric optimization leads to

substantial improvements in fidelity, robustness, and generalization. More broadly, this work highlights the importance of integrating generative models with search and optimization for structured program synthesis tasks. We believe this paradigm can be extended beyond CAD reconstruction to applications such as image-to-CAD generation, interactive design, and automated engineering workflows.

References

- [1] Ahmet Serdar Karadeniz, Dimitrios Mallis, Danila Rukhovich, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. MiCADangelo: Fine-grained reconstruction of constrained CAD models from 3d scans. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [2] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9801–9811, 2025.
- [3] Hongyuan Ren, Bo Xia, Wenrui Wang, Xueqian Chen, Yang Liu, Pingzhang Zhou, and Jianbin Du. Amrto: Automated cad model reconstruction of topology optimization result. *Computer Methods in Applied Mechanics and Engineering*, 435:117673, 2025.
- [4] Abdenmour Amroune, Jean-Christophe Cuilliere, and Vincent Francois. Automated lofting-based reconstruction of cad models from 3d topology optimization results. *Computer-Aided Design*, 145:103183, 2022.
- [5] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21469–21480, 2025.
- [6] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, Yunfei Zhao, Haolin Liu, Shuhui Yang, Yifei Feng, Mingxin Yang, Sheng Zhang, Xianghui Yang, et al. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation. *arXiv preprint arXiv:2501.12202*, 2025.
- [7] Yuan Li, Cheng Lin, Yuan Liu, Xiaoxiao Long, Chenxu Zhang, Ningna Wang, Xin Li, Wenping Wang, and Xiaohu Guo. Caddreamer: Cad object generation from single-view images. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21448–21457, 2025.
- [8] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [9] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G Lambourne, Karl DD Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation. *arXiv preprint arXiv:2307.00149*, 2023.
- [10] Maksim Kolodiaznyi, Denis Tarasov, Dmitrii Zhemchuzhnikov, Alexander Nikulin, Ilya Zisman, Anna Vorontsova, Anton Konushin, Vladislav Kurenkov, and Danila Rukhovich. cadrille: Multi-modal cad reconstruction with online reinforcement learning. *arXiv preprint arXiv:2505.22914*, 2025.
- [11] Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodality-conditioned cad generation with mllm. *arXiv preprint arXiv:2411.04954*, 2024.
- [12] Mohammadmehdi Ataei, Farzaneh Askari, Kamal Rahimi Malekshan, and Pradeep Kumar Jayaraman. Zero-to-cad: Agentic synthesis of interpretable cad programs at million-scale without real data. *arXiv preprint arXiv:2604.24479*, 2026.
- [13] Matej Balog, Alexander L Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. *arXiv preprint arXiv:1611.01989*, 2016.
- [14] Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a repl. *Advances in Neural Information Processing Systems*, 32, 2019.
- [15] Xinyun Chen, Chang Liu, and Dawn Song. Execution-guided neural program synthesis. In *International Conference on Learning Representations*, 2018.

- [16] Lyle Regenwetter, Cyril Picard, Amin Heyrani Nobari, Akash Srivastava, and Faez Ahmed. Generative optimization: A perspective on ai-enhanced problem solving in engineering. *arXiv preprint arXiv:2412.13281*, 2024.
- [17] Anna C. Doris, Jacob Thomas Sony, Ghadi Nehme, Era Sylva, Amin Heyrani Nobari, and Faez Ahmed. CADBench: A multimodal benchmark for AI-assisted CAD program generation. *arXiv preprint arXiv:2605.10873*, 2026.
- [18] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9601–9611, 2019.
- [19] Karl DD Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4):1–24, 2021.
- [20] Sangpil Kim, Hyung-gun Chi, Xiao Hu, Qixing Huang, and Karthik Ramani. A large-scale annotated mechanical components benchmark for classification and retrieval tasks with deep neural networks. In *European conference on computer vision*, pages 175–191. Springer, 2020.
- [21] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13142–13153, 2023.
- [22] Changqi He, Shuhan Zhang, Liguang Zhang, and Jiajun Miao. Cad-coder: Text-guided cad files code generation. *arXiv preprint arXiv:2505.08686*, 2025.
- [23] Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. Text-to-cad generation through infusing visual feedback in large language models. *arXiv preprint arXiv:2501.19054*, 2025.
- [24] Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579, 2024.
- [25] Yujia Liu, Anton Obukhov, Jan Dirk Wegner, and Konrad Schindler. Point2cad: Reverse engineering cad models from 3d point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3763–3772, 2024.
- [26] Shane Larsen and C Greg Jensen. Converting topology optimization results into parametric cad models. *Computer-Aided Design and Applications*, 6(3):407–418, 2009.
- [27] Nomi Yu, Md Ferdous Alam, A John Hart, and Faez Ahmed. Gencad-3d: Cad program generation using multimodal latent space alignment and synthetic dataset balancing. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89220, page V03AT03A015. American Society of Mechanical Engineers, 2025.
- [28] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. *arXiv preprint arXiv:2207.04632*, 2022.
- [29] Cheng Chen, Jiacheng Wei, Tianrun Chen, Chi Zhang, Xiaofeng Yang, Shangzhan Zhang, Bingchen Yang, Chuan-Sheng Foo, Guosheng Lin, Qixing Huang, et al. Cadcrafter: Generating computer-aided design models from unconstrained images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11073–11082, 2025.
- [30] Anna C Doris, Md Ferdous Alam, Amin Heyrani Nobari, and Faez Ahmed. Cad-coder: An open-source vision-language model for computer-aided design code generation. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89220, page V03AT03A031. American Society of Mechanical Engineers, 2025.

- [31] Soslan Kabisov, Vsevolod Kirichuk, Andrey Volkov, Gennadii Savrasov, Marina Barannikov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadreasoner: Iterative program editing for cad reverse engineering. *arXiv preprint arXiv:2603.29847*, 2026.
- [32] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*, pages 26106–26128. PMLR, 2023.
- [33] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- [34] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*, 2023.
- [35] Xiang Xu, Joseph Lambourne, Pradeep Jayaraman, Zhengqing Wang, Karl Willis, and Yasutaka Furukawa. Brepger: A b-rep generative diffusion model with structured latent geometry. *ACM Transactions on Graphics (TOG)*, 43(4):1–14, 2024.
- [36] Mingi Lee, Dongsu Zhang, Clément Jambon, and Young Min Kim. Brepdiff: Single-stage b-rep diffusion model. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pages 1–11, 2025.
- [37] Mikaela Angelina Uy, Yen-Yu Chang, Minhyuk Sung, Purvi Goel, Joseph G Lambourne, Tolga Birdal, and Leonidas J Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11850–11860, 2022.
- [38] Tao Du, Jeevana Priya Inala, Yewen Pu, Andrew Spielberg, Adriana Schulz, Daniela Rus, Armando Solar-Lezama, and Wojciech Matusik. Inversecsg: Automatic conversion of 3d models to csg trees. *ACM Transactions on Graphics (TOG)*, 37(6):1–16, 2018.
- [39] Maksim Elistratov, Marina Barannikov, Gregory Ivanov, Valentin Khrulkov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadevolve: Creating realistic cad via program evolution. *arXiv preprint arXiv:2602.16317*, 2026.
- [40] CadQuery contributors. CadQuery. <https://doi.org/10.5281/zenodo.14590990>, October 2025. <https://doi.org/10.5281/zenodo.14590990>.
- [41] Tianrun Chen, Chunan Yu, Yuanqi Hu, Jing Li, Tao Xu, Runlong Cao, Lanyun Zhu, Ying Zang, Yong Zhang, Zejian Li, et al. Img2cad: Conditioned 3-d cad model generation from single image with structured visual geometry. *IEEE Transactions on Industrial Informatics*, 2025.
- [42] Yatian Pang, Eng Hock Francis Tay, Li Yuan, and Zhenghua Chen. Masked autoencoders for 3d point cloud self-supervised learning. *World Scientific Annual Review of Artificial Intelligence*, 1:2440001, 2023.
- [43] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- [44] Manuel Contero, David Pérez-López, Pedro Company, and Jorge D Camba. A quantitative analysis of parametric cad model complexity and its relationship to perceived modeling complexity. *Advanced Engineering Informatics*, 56:101970, 2023.
- [45] Yujia Zhang, Xiaoyang Wu, Yunhan Yang, Xianzhe Fan, Han Li, Yuechen Zhang, Zehao Huang, Naiyan Wang, and Hengshuang Zhao. Utonia: Toward one encoder for all point clouds. *arXiv preprint arXiv:2603.03283*, 2026.

Table of Contents for Appendices

A	Ablations	16
A.1	Which Components of II-StepCAD Matter Most?	16
A.2	How Do Model Scale and Geometry Encoders Affect Performance?	16
A.3	Which Local Edits Contribute Most to Geometry-Aware Search?	16
B	StepCAD: Applications	17
B.1	Variants Generation	17
B.2	Image-to-CAD	17
C	One-to-Many Mapping: From Geometry to CAD Construction Sequence	18
D	Analysis of Generated CAD Operation Distributions	18
E	Metrics	19
F	Dataset Statistics	20
G	Dataset Generation Details	20
G.1	Sketch Database	20
G.2	Sketch Placement and Scaling	20
G.3	Primitive Code Generation	21
G.4	SolidFactory: Bounding-Box-Driven Dispatch	22
G.5	ModelBuilderV2: Composing Primitives into Parts	22
G.6	Code Assembly and Post-Processing	23
G.7	Strategy Library	24
G.8	Parallel Generation and Double Validation	24
G.9	Dataset Characteristics	24
H	Policy Architecture and Training Details	26
H.1	Inputs	26
H.2	Point Cloud Encoding	26
H.3	Geometry Conditioning	26
H.4	Multimodal Token Fusion	26
H.5	Language Model	26
H.6	Decoding and Execution	26

H.7	Training Setup	26
H.8	Additional StepCAD Details	27
H.8.1	Problem Formulation	27
H.8.2	Autoregressive Policy Formulation	27
H.8.3	Dense State-Action Supervision	27
H.8.4	Closed-Loop Rollout Procedure	28
H.8.5	Motivation for Program Refinement	28
I	Geometry-Aware Program Optimization Details	29
I.1	Beam Search Procedure	29
I.2	Continuous Parameter Refinement	29
I.3	Sketch Extraction and Replacement	30
I.4	Operation Skipping	30
I.5	Complement-Cut Optimization	30

A Ablations

A.1 Which Components of Π -StepCAD Matter Most?

Table 4: **Ablation study of StepCAD.** CD $\downarrow$ and IoU $\uparrow$ evaluate reconstruction quality; VSR denotes valid shape rate.

Variant	CD $\downarrow$	IoU $\uparrow$	VSR (%) $\uparrow$
Π -StepCAD w/o prompt conditioning	–	–	0
Π -StepCAD w/o FPS point sampling	0.209	0.105	100
Π -StepCAD w/o input geometry tokens	0.144	0.209	100
Π -StepCAD w/o Point-MAE fine-tuning	0.093	0.316	100
CADEvolve	0.056	0.568	97
Π -StepCAD w/o previous-geometry tokens	0.045	0.632	100
Π-StepCAD	0.028	0.682	100
StepCAD	0.025	0.816	100

A.2 How Do Model Scale and Geometry Encoders Affect Performance?

Table 5: **Ablation study for model size and point cloud encoder for Π -StepCAD.** Best validation loss achieved within the first 10K training steps. Lower is better.

Variant	Best Val Loss at 10K steps $\downarrow$
Π -StepCAD (0.5B + Point-MAE)	0.3559
Π -StepCAD (1.5B + Point-MAE)	<u>0.3433</u>
Π-StepCAD (3B + Point-MAE)	0.3419
Π -StepCAD (1.5B + Utonia [45])	0.3450

Although the 3B model achieves the lowest early validation loss, its gain over the 1.5B model is marginal (0.0014), so we use the 1.5B Point-MAE configuration as the default due to its favorable compute-performance trade-off.

A.3 Which Local Edits Contribute Most to Geometry-Aware Search?

Table 6: **Local edit-level ablation of the tree search.** Mean metrics;

Win-rate: fraction of beam steps where this phase’s output is the rank-0 candidate (i.e. the new beam[0]).

Realized Δ IoU: average per-sample IoU lift attributed to this phase via winner-attribution at each step; the four phase rows sum to the *Full search* row by construction.

% share: the phase’s Realized Δ divided by the Full Realized Δ .

Cost: mean wall-time per sample.

Local Edit	Win-rate	Realized Δ	% share	Cost (s)
Sketch Modification	26.0%	+0.059	41.5%	12.5
Complement Cut	24.4%	+0.043	30.3%	17.0
Continuous refinement	36.6%	+0.023	16.2%	28.4
Operation skipping	13.0%	+0.017	12.0%	8.1
Full search	100%	+0.142	100%	66.0

B StepCAD: Applications

B.1 Variants Generation

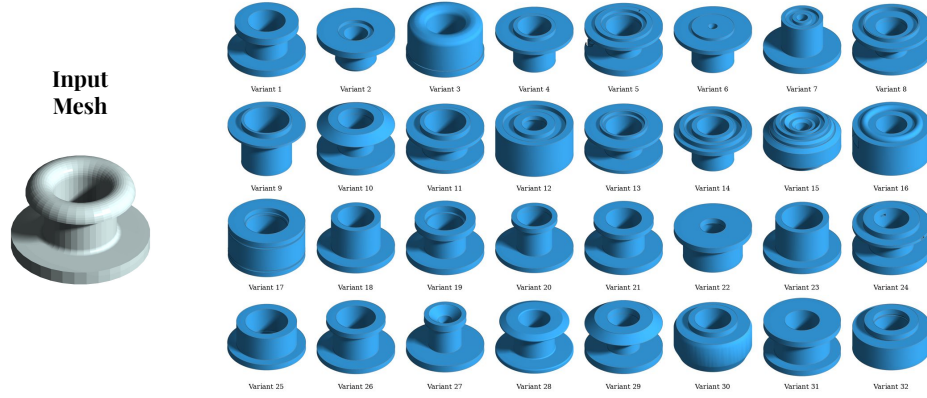


Figure 4: **Design variation generation with II-StepCAD.** Given an input mesh, II-StepCAD can generate diverse parametric CAD variants that preserve the overall structure while modifying geometric details and construction choices. This demonstrates a potential application of the learned policy for CAD design exploration and rapid generation of editable shape variations from a single reference geometry.

B.2 Image-to-CAD

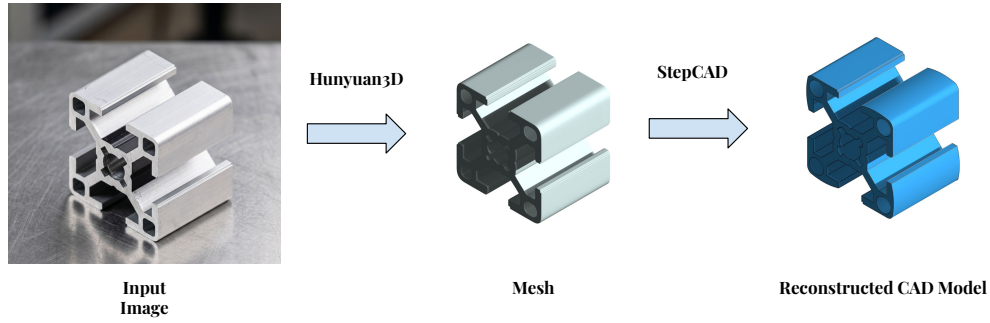


Figure 5: **Image-to-CAD reconstruction using StepCAD.** An input image is first converted into a 3D mesh using Hunyuan3D [6], after which StepCAD reconstructs an editable parametric CAD program from the generated geometry. This demonstrates how StepCAD can be integrated into image-to-CAD pipelines to transform unstructured visual inputs into structured CAD representations.

C One-to-Many Mapping: From Geometry to CAD Construction Sequence

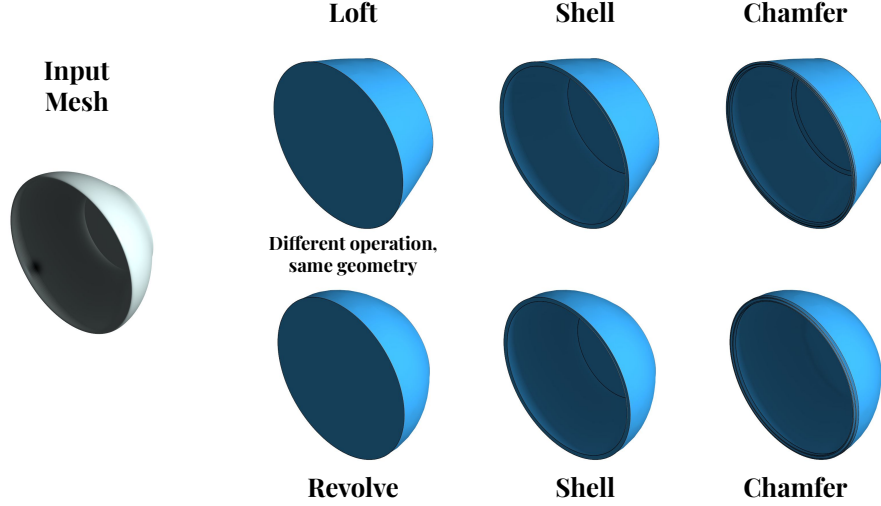


Figure 6: **One-to-many CAD program generation from a single input mesh.** Given the same target geometry, StepCAD generates multiple valid CAD programs with different operation sequences. In this example, the model reconstructs the shape using either a loft-based or revolve-based strategy, followed by shell and chamfer operations, producing nearly identical final geometry. This demonstrates that the policy learns a distribution over valid CAD construction procedures rather than a single deterministic reconstruction.

D Analysis of Generated CAD Operation Distributions

Table 7: **Operation occurrence across generated programs (%)**. Values indicate the percentage of generated CAD programs that contain at least one instance of the given operation.

Model	Extrude	Revolve	Sweep	Loft	Fillet	Chamfer	Shell	Mirror
StepCAD	89.35	16.86	7.75	11.48	9.60	6.24	5.11	2.82
CADEvolve	81.10	0.20	0.00	1.20	3.30	1.80	0.20	0.00
Cadrille	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
CAD-Recode	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Table 8: **Operation occurrence across generated programs (%)**. Values indicate the percentage of generated CAD programs that contain at least one instance of the given operation.

Benchmark	Model	Extrude	Revolve	Sweep	Loft	Fillet	Chamfer	Shell	Mirror
DeepCAD	StepCAD	94.5	11.3	4.4	4.9	4.1	1.7	2.9	2.2
	CADEvolve	80.9	0.0	0.0	0.0	0.1	0.0	0.4	0.0
Fusion360	StepCAD	86.7	19.9	10.2	12.5	10.0	4.9	3.3	3.1
	CADEvolve	81.7	0.0	0.0	0.8	3.1	1.0	0.1	0.0
ABC	StepCAD	92.0	14.4	6.1	8.4	9.9	5.5	7.4	2.7
	CADEvolve	82.9	0.0	0.0	0.6	4.8	1.3	0.2	0.0
Mechanical	StepCAD	75.7	33.0	6.1	32.2	23.3	17.6	5.3	2.3
	CADEvolve	70.2	0.0	0.0	2.5	5.0	6.8	0.0	0.0
Organic	StepCAD	79.6	23.2	21.8	30.4	12.9	20.4	6.5	4.8
	CADEvolve	88.1	0.8	0.1	3.0	1.9	0.6	0.2	0.0

E Metrics

We report the following metrics:

- **Intersection-over-Union (IoU):** volumetric overlap between predicted and target shapes.

$$\text{IoU} = \frac{V(\hat{S} \cap S)}{V(\hat{S} \cup S)}$$

- **Chamfer Distance (CD):** surface distance between point clouds (lower is better).

$$\text{CD}(\hat{P}, P) = \frac{1}{2} \left(\frac{1}{|\hat{P}|} \sum_{x \in \hat{P}} \min_{y \in P} \|x - y\|_2^2 + \frac{1}{|P|} \sum_{y \in P} \min_{x \in \hat{P}} \|y - x\|_2^2 \right)$$

- **Valid Shape Rate (VSR):** percentage of generated programs that execute successfully and produce valid geometry.

F Dataset Statistics

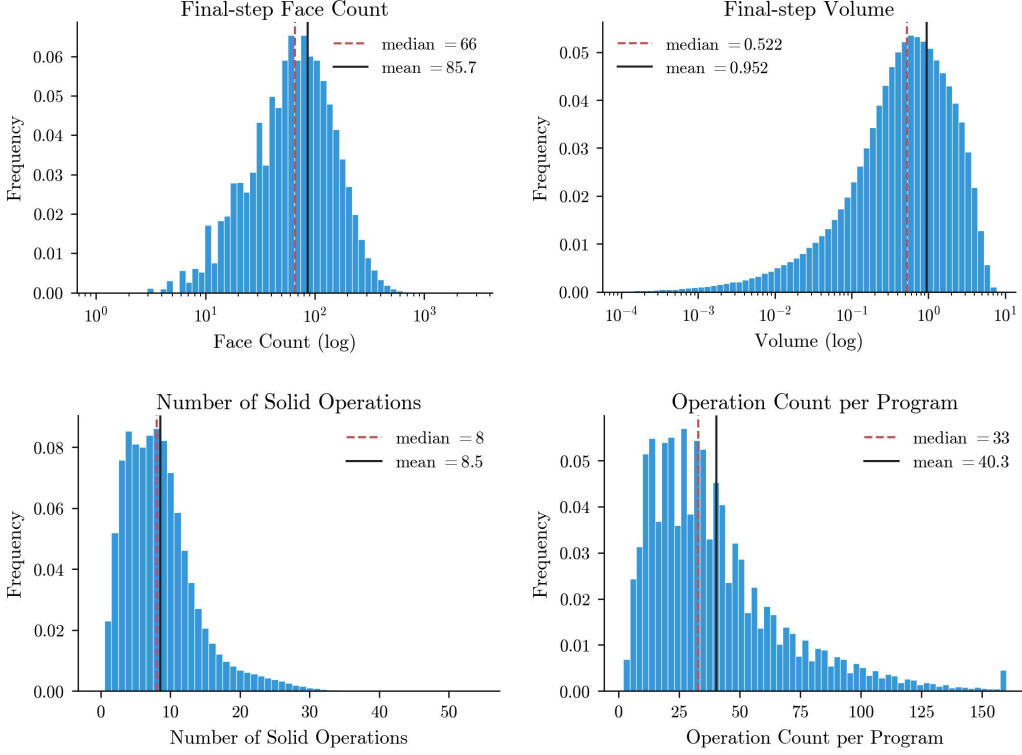


Figure 7: **ARCADE-1.5M dataset statistics.** We report distributions of face count, volume, solid operations and total operations per program. The dataset spans a wide range of shapes and scales, while exhibiting long-horizon construction sequences and non-trivial solid modeling depth. Operation counts range from 3 to 150+, with a median of 33 and a mean of 40.3. Dashed and solid lines denote median and mean, respectively.

G Dataset Generation Details

G.1 Sketch Database

The fundamental source of geometric diversity is a curated database of 2-D sketch loop primitives derived from real engineering CAD models. Each sketch entry is a dictionary containing a list of *primitives*, where every primitive is one of four types:

- **Line:** a pair of (x, y) endpoints.
- **Arc:** three control points (start, mid, end) defining a circular arc, encoded as a `threePointArc`.
- **Circle:** a centre point and a radius.
- **Spline:** an ordered list of (x, y) control points defining a B-spline curve.

At runtime, `CADFeatureTreeGenerator.get_random_sketch()` draws a uniformly random entry from this index and loads it on demand, with an in-memory LRU cache to avoid repeated disk I/O.

G.2 Sketch Placement and Scaling

Before a sketch is used to build a solid it must be fitted into a caller-specified 3-D axis-aligned bounding box (*bbbox*). The procedure is:

1. **Compute sketch 2-D bbox.** All primitive control points are collected and the tight axis-aligned bounding box $[s_x^{\min}, s_y^{\min}, s_x^{\max}, s_y^{\max}]$ is computed.
2. **Select a face of the target 3-D bbox.** The six bbox faces are enumerated with their outward normals and local x -directions. One face is chosen uniformly at random as the workplane for the sketch.
3. **Compute uniform scale and 2-D translation.** The sketch is uniformly scaled so that it fills a random fraction f of the face's 2-D extent, then translated so its centre coincides with the face centre:
$$s = f \cdot \min\left(\frac{2T_w}{S_w}, \frac{2T_h}{S_h}\right), \quad c_x = \frac{T_x^{\min} + T_x^{\max}}{2}, \quad c_y = \frac{T_y^{\min} + T_y^{\max}}{2}.$$
4. **Apply transform to primitives.** Every control point (p_x, p_y) is mapped to $(p_x \cdot s + c_x, p_y \cdot s + c_y)$ before being written into the generated code.

G.3 Primitive Code Generation

Given a placed-and-scaled sketch and a target 3-D bbox, `generate_cadquery_code()` emits CadQuery Python code for one of four volumetric operations.

Extrude. A named workplane is created from `cq.Plane(origin=..., normal=..., xDir=...)`, aligned with the chosen bbox face. The sketch primitives are emitted in order: lines as `.moveTo(x0,y0).lineTo(x1,y1)` (first) or `.lineTo(x1,y1)` (subsequent); arcs as `.threePointArc((x1,y1),(x2,y2))`; circles as `.moveTo(cx,cy).circle(r)`; splines as `.spline([(x0,y0),...])`. A `.close()` seals the contour and `.extrude(d)` creates the solid, where d fills 85–100% of the bbox dimension normal to the face. Example output:

```
plane_0 = cq.Plane(origin=(0,0,0), normal=(0,0,1), xDir=(1,0,0))
sketch_0 = cq.Workplane(plane_0)
sketch_0 = sketch_0.moveTo(-1.2,-0.8).lineTo(1.2,-0.8)
sketch_0 = sketch_0.threePointArc((1.5,0.0),(1.2,0.8))
sketch_0 = sketch_0.lineTo(-1.2,0.8).close()
solid_0 = sketch_0.extrude(2.5)
```

Revolve. Workplane and sketch emission are identical to extrude. After `.close()`, the solid is created with `.revolve(angleDegrees= $\theta$ , axisStart=(ax,ay), axisEnd=(bx,by))`, where $\theta \in \{90, 180, 270, 360\}$ is chosen uniformly and the axis is either a line extracted from the sketch primitives (50% probability) or an edge of the face bbox. With 25% probability a circle replaces the database sketch profile.

Sweep (spline path). A 2-D spline path is generated by sampling one of ten named archetypes — *open*: arc, S-curve, U-curve, L-curve, wave, diagonal; *closed*: ellipse, rounded rectangle, teardrop, kidney — and emitted as:

```
path_0 = cq.Workplane('XY').spline([(x0,y0),(x1,y1),...])
```

(with `.close()` for closed loops). The profile cross-section on the start face is emitted identically to the extrude case, then swept with `profile_0.sweep(path_0)`.

Loft (offset). A single workplane is created. The sketch is drawn $N \in [3, 7]$ times at successive `.workplane(offset= $\delta_i$ )` steps, each with a per-profile scale factor that creates a tapered or gradually varying cross-section. The chain is closed with `.loft()`.

Loft (transformed). The base profile is drawn on the start workplane. Each subsequent profile is reached with `.transformed(offset=( $\Delta x, \Delta y, \Delta z$ ), rotate=( $r_x, r_y, r_z$ ))`, where offsets follow a 2-D spline spine (same archetypes as sweep) and rotation angles are drawn from the discrete set $\{0, 10, 20, 30, 45, 60, 90\}^\circ$. Profile scale also varies per section. `.loft()` is called after all profiles are drawn.

Per-primitive fillet/chamfer. With 50% probability, each primitive solid is immediately followed by `solid_N = solid_N.edges().fillet(r)` or `.chamfer(d)`, $r, d \in [0.1, 0.9]$, applied as part of the primitive's own code block before it enters the Boolean tree.

G.4 SolidFactory: Bounding-Box-Driven Dispatch

SolidFactory wraps CADFeatureTreeGenerator and exposes a uniform bounding-box interface: every public method accepts $(c_x, c_y, c_z, s_x, s_y, s_z)$ (centre + extents) and returns a validated CadQuery solid together with the code string that built it. The internal flow is:

1. Convert centre/extents to a bbox dict.
2. Call the appropriate `generate_*` method. The generator selects a face, scales the sketch, and returns a feature dict.
3. Pass the feature dict to `generate_cadquery_code()`, which produces a Python string.
4. `exec()` the string and extract `final_result`.
5. Validate: single body, volume > 0.01, B-Rep valid (`BRepCheck_Analyzer.IsValid()`).
6. Strip the import header and `final_result` line; store the remaining code in `self._last_code` and the variable name in `self._last_var`.

Fallback hierarchy on failure: sketch-based extrude → cylinder → axis-aligned box.

G.5 ModelBuilderV2: Composing Primitives into Parts

ModelBuilderV2 is a stateful accumulator maintaining: `self.solid` (the current CadQuery solid), `self._code_lines` (a growing list of code-string fragments), and `self._model_var` (the Python variable name of the current combined solid).

At each step an operation is attempted in memory. If it produces a valid single-body solid, the operation's code fragment is appended and the solid updated; otherwise the step is silently skipped, making the builder failure-tolerant.

Base creation. `create_base()` calls SolidFactory with one of the four primitive types (or all four in random order until one succeeds) to create the initial solid, centred at the origin with depth extending along +Z. `create_nested_base()` builds a solid from an *outer* polygon profile with one or more *inner* cutout profiles (simulating perforated cross-sections such as frames or flanged tubes).

Union bosses. `add_boss()` / `add_boss_ex()` place a new primitive on a named face (top, bottom, front, back, left, right). `_face_region()` maps fractional width/height/depth parameters and a lateral offset (u, v) into a 3-D bbox, with an outward overlap to anchor the boss to the face surface. The primitive is generated inside that box and boolean-unioned:

```
result = result.union(solid_N)
```

`add_step()` / `add_multi_step()` place wide, shallow bosses covering progressively smaller face fractions to build stepped profiles.

Intersection constraints for union bosses. All boss placement functions pass an `overlap_frac` parameter to `_face_region()`, which pushes the boss bbox inward by that fraction of the face depth so the boss is geometrically anchored to the host solid rather than merely touching it. The overlap fraction is sampled as $[0.05, 0.20]$ for regular bosses and $[0.05, 0.15]$ for steps. This is a placement-level geometric constraint (bbox overlap), not a volumetric ratio check.

Boolean cuts. `add_pocket()` and `add_through_cut()` compute an inward face region, generate a tool (90% database-sketch extrude, 5% cylinder, 5% rect box), then apply a volumetric intersection constraint before committing: `has_sufficient_intersection()` computes

$$\frac{V(\text{tool} \cap \text{solid})}{V(\text{tool})} \geq 0.20$$

and rejects the cut if the ratio falls below 20%, preventing tools that barely clip the surface.

```
result = result.cut(solid_N)
```

Through-cuts use a tool depth of $1.2\times$ the solid's full depth to guarantee complete penetration. `add_slot()` produces a narrow elongated cut (aspect ratio $\sim 6:1$). `add_nested_cut()` subtracts a nested outer-with-inner profile tool.

Spatial patterns. All three pattern types emit for-loop code.

Linear pattern — equally spaced tools along one face axis:

```
for i in range(N):
    pos = base + step * (i + 0.5)
    plane_i = cq.Plane(origin=(pos, cy, z0), ...)
    sketch_i = cq.Workplane(plane_i)
    sketch_i = sketch_i.circle(r)          # or polygon
    pat_tool = sketch_i.extrude(h)
    result = result.cut(pat_tool)         # or .union()
```

Circular pattern — N equally-spaced tools on a circle of radius ρ centred at (c_u, c_v) :

```
for i in range(N):
    angle = phi0 + 2*math.pi*i/N
    fu = cu + rho*math.cos(angle)
    fv = cv + rho*math.sin(angle)
    ...
```

Grid pattern — doubly-nested loop over rows $\times$ cols:

```
for ir in range(rows):
    for ic in range(cols):
        pu = bu + su*(ic+0.5)
        pv = bv + sv*(ir+0.5)
    ...
```

Each pattern commits only if ≥ 2 individual tool applications succeed.

Mirror. `add_mirror()` reflects the solid across XY, XZ, or YZ and unions the reflection. With 50% probability an off-centre mirror plane is used via a `basePointVector`.

Shell. `add_shell()` removes a selected face and hollows the solid to a specified wall thickness:

```
result = result.faces('>Z').shell(-t)
```

Intermediate fillet/chamfer. After union and cut operations, a 25–50% probability inline fillet or chamfer is applied to the sub-solid before the next operation, simulating mid-process edge-breaking in machining. Radius/distance is 1–4% of the current solid’s minimum dimension.

Final fillet/chamfer. `apply_fillet()` and `apply_chamfer()` are called at the end of a strategy, with radius/distance drawn as 2–20% of the minimum bounding-box dimension. On failure a smaller value (50%, 25%, or 10% of original) is tried automatically.

G.6 Code Assembly and Post-Processing

After construction, `get_code()` assembles the final self-contained Python script as follows:

1. Prepend `import cadquery as cq; import math.`
2. Join all `_code_lines` fragments in sequence.
3. Append `final_result = result` and `cq.exporters.export(final_result, 'model.step')`.
4. **Variable renumbering** (`_renumber_vars`): `plane_N`, `sketch_N`, `solid_N`, `loft_N`, `profile_N`, `path_N` indices are made contiguous. Gaps arise because the factory counter increments even on failed attempts.
5. **Variable substitution** (`_substitute_result_var`): `solid_0` is renamed to `result`; intermediate bare `result` assignments are removed.
6. **Float rounding** (optional, `-round-floats`): every float literal is rounded to three decimal places via regex substitution, reducing token count for language-model training.

7. **Collinear point removal** (`_remove_collinear_points`): any `lineTo` point that lies between its predecessor and successor on the same line, or duplicates either endpoint, is eliminated. Chained `.moveTo().lineTo()` calls are first split onto separate lines so that rounding-induced duplicates are also caught.

G.7 Strategy Library

Ninety-four named strategy functions orchestrate the builder, organised into seven tiers by complexity and intent:

- Tier 1 — Single-primitive (1–4).** One primitive type forms the entire body (`strategy_extruded_block`, `strategy_revolved_body`, `strategy_lofted_body`, `strategy_swept_body`).
- Tier 2 — Mid-level composition (5–20).** A primary primitive receives 2–6 additional features (bosses, pockets, patterns, mirror, shell). Examples: `strategy_symmetric_extruded` (`extrude` → features on one half → mirror), `strategy_lofted_enclosure` (`loft` → shell → mounting bosses).
- Tier 3 — Higher-level composition (21–30).** 10–18-feature sequences mimicking specific mechanical part families.
- Tier 4 — Union-focused (31–40).** Emphasise additive material: multi-scale boss clusters, tall fin arrays, nested cylinder rings, pancake stacks, hub-and-arm assemblies.
- Tier 5 — Mega-composite (41–48).** Combine two or more mid-level sub-strategies within one builder instance, yielding 12–25 features.
- Tier 6 — Long-sequence (49–72).** Detail-pass and nested-sketch strategies targeting 20–50+ features: surface nubs, micro-pockets, screw-boss panels, multi-hole plates, nested-sketch towers.
- Tier 7 — Operation-subset boost (73–94).** Each strategy restricts itself to underrepresented operations to rebalance the operation-type distribution: revolve-only, loft-only, sweep-only, cut-dominant, shell+fillet+chamfer focus, circular-pattern-only, mirror-heavy, sweep/loft alternating, grid/slot textured, revolve-cut lathe, circular-union crown, linear-union array.

G.8 Parallel Generation and Double Validation

Worker architecture. Generation is parallelised across W workers (default: all CPU cores). Each worker is an isolated subprocess spawned via `subprocess.Popen` rather than `multiprocessing.Process`, because forking from a `ThreadPoolExecutor` thread is unsafe with OpenCASCADE’s global C++ state. A per-model timeout (default: 120 s) sends `SIGKILL` to workers that hang inside the kernel.

Double validation. Each candidate model undergoes two independent validity checks:

1. **In-memory validation.** The builder solid must satisfy: (a) exactly one connected body (`count_solids=1`), (b) `volume > 0.01`, (c) `BRepCheck_Analyzer.IsValid()` returns `True`.
2. **Script re-execution validation.** The generated `.py` script is independently `exec()`-ed in a fresh namespace; the resulting solid must pass the same three checks. The STEP file is exported from *this* re-executed solid, guaranteeing that the stored script and the stored geometry are identical.

File management. Successful models are stored in chunk subdirectories of 10 000 files each (`0000/`, `0001/`, ...). Partial files from failed workers are cleaned up before committing. Completed runs write `metadata.json` (strategy, random seed, geometric scale, full feature history with parameters, axis-aligned bounding box, volume) and `strategy_stats.json` (per-strategy success/invalid/timeout/error counts).

G.9 Dataset Characteristics

Artifact types per model.

- `model_XXXXX.py`: self-contained CadQuery script (ground-truth program).
- `model_XXXXX.step`: STEP B-Rep geometry (exported from the re-executed script).
- `model_XXXXX.stl`: triangulated mesh (optional).

Program characteristics. Scripts consist of sequences of CadQuery API calls with explicit workplane definitions (`cq.Plane`), sketch primitives (`moveTo`, `lineTo`, `threePointArc`, `circle`, `spline`, `close`), volumetric operations (`extrude`, `revolve`, `loft`, `sweep`), Boolean combinators (`union`, `cut`), topology operations (`mirror`, `shell`), finishing operations (`fillet`, `chamfer`), and for-loop spatial patterns. Float literals are rounded to three decimal places.

H Policy Architecture and Training Details

H.1 Inputs

At each step, the model receives:

- **Target geometry** m : 1024-point cloud sampled from the input mesh.
- **Intermediate geometry** S_{t-1} : 1024-point cloud from the current reconstruction.
- **Geometry metadata**: bounding box center and scale of S_{t-1} in the target frame.
- **Program context**: the partial CAD program encoded as text tokens.

H.2 Point Cloud Encoding

Both target and intermediate point clouds are encoded using a Point-MAE encoder:

- Farthest Point Sampling: $1024 \rightarrow 64$ groups
- KNN grouping ($k = 32$)
- PointNet-based per-group encoder
- Transformer encoder (12 layers, partial fine-tuning)

This produces 64 tokens per point cloud, which are projected to the LLM embedding space.

H.3 Geometry Conditioning

A lightweight MLP encodes geometric metadata (center + scale) into a token that preserves global pose information lost during normalization.

H.4 Multimodal Token Fusion

Geometric tokens are injected into the language model by replacing reserved placeholder tokens in the input sequence. This yields a unified embedding stream combining:

- target geometry tokens
- intermediate geometry tokens
- geometric metadata
- CAD program text

H.5 Language Model

We use Qwen2.5-Coder-1.5B as the backbone. The model autoregressively predicts the next CAD operation as cadquery code, including continuous parameters represented as digit sequences. Prompt: “Generate CAD operation $\{\text{op}_{i+1}\}$ for this shape:”

H.6 Decoding and Execution

At inference time:

- The model generates one operation at a time
- The operation is executed to produce updated geometry
- The resulting geometry is re-encoded for the next step

H.7 Training Setup

- Cross-entropy loss over operation tokens
- Full fine-tuning of the language model

- Separate learning rates for point cloud encoder (0.00005) and LLM (0.0002)
- 1000 learning-rate warmup steps followed by cosine decay
- Training on $\sim 12.5\text{M}$ state-action transitions
- Training performed on 8 NVIDIA H100 GPUs for 12 hours, batch size of 32, for 100k steps

H.8 Additional StepCAD Details

H.8.1 Problem Formulation

Let $m \in \mathcal{M}$ denote an input mesh, and let $Q_m \in \mathbb{R}^{N \times 3}$ denote the corresponding sampled point cloud. A CAD program is represented as

$$p = (a_1, \dots, a_T),$$

where $a_t \in \mathcal{A}$ are executable CAD operations and $S(p)$ is the resulting geometry. The reconstruction objective is:

$$p^* = \arg \max_{p \in \mathcal{P}} \text{IoU}(S(p), m),$$

where $\mathcal{P}$ denotes the space of valid CAD programs.

Direct optimization over $\mathcal{P}$ is combinatorial and highly non-convex due to the discrete, sequential, and structured nature of CAD construction programs.

H.8.2 Autoregressive Policy Formulation

We model CAD generation as a sequential decision process. Let Q_t denote the point cloud sampled from the intermediate geometry S_t . At each step, the policy predicts:

$$\pi_\theta(a_t \mid Q_{t-1}, Q_m, a_{<t}),$$

which induces a distribution over complete programs:

$$\pi_\theta(p) = \prod_{t=1}^T \pi_\theta(a_t \mid Q_{t-1}, Q_m, a_{<t}).$$

Target and intermediate geometries are encoded using a shared Point-MAE encoder:

$$z_m = f_\phi(Q_m), \quad z_{t-1} = f_\phi(Q_{t-1}),$$

which are injected into the transformer as conditioning tokens.

The policy is instantiated using Qwen2.5-Coder-1.5B operating jointly over geometry embeddings and autoregressive program tokens. The next operation is decoded as:

$$a_t = \arg \max_a \pi_\theta(a \mid z_m, z_{t-1}, a_{<t}),$$

or sampled from the predictive distribution during inference-time sampling.

H.8.3 Dense State-Action Supervision

The policy is trained using teacher forcing over dense state-action transitions extracted from ARCADE-1.5M:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(Q_{t-1}, Q_m, a_t)} [\log \pi_\theta(a_t \mid Q_{t-1}, Q_m, a_{<t})].$$

Unlike supervision only on final programs, ARCADE-1.5M provides intermediate geometry after every operation, yielding dense trajectory-level supervision over approximately 12.5M construction states. This improves long-horizon robustness and geometric consistency.

H.8.4 Closed-Loop Rollout Procedure

At inference time, generation alternates between autoregressive prediction and CAD execution. Starting from an empty geometry $S_0 = \emptyset$, at each step:

1. The current intermediate geometry Q_{t-1} and target geometry Q_m are encoded.
2. The next operation is sampled from

$$a_t \sim \pi_\theta(\cdot \mid Q_{t-1}, Q_m, a_{<t}),$$

using inference-time sampling.

3. The updated program is executed using CadQuery to obtain the next geometry:

$$S_t = \text{Exec}(a_1, \dots, a_t).$$

4. A new point cloud Q_t is sampled from S_t and fed back into the policy.
5. Reconstruction quality is evaluated using:

$$r_t = \text{IoU}(S_t, m).$$

Generation terminates once reconstruction quality plateaus ($\epsilon < 0.001$), an end-of-program token is emitted, or a maximum sequence length is reached ($T_{max} = 12$).

This closed-loop execution is important because the policy conditions on the geometry actually produced so far rather than on teacher-forced intermediate states.

H.8.5 Motivation for Program Refinement

The learned policy efficiently captures the global structure of target shapes. However, due to the autoregressive and discretized nature of generation, small structural or parametric errors can accumulate over long construction sequences, motivating the geometry-aware refinement stage described in the main paper.

Algorithm 1 Geometry-aware tree search for CAD program refinement.

Require: Target mesh m , seed program $\hat{p} = (a_1, \dots, a_T)$, beam width K , local edits $\{e_1, \dots, e_M\}$

```
1:  $\mathcal{B}_0 \leftarrow \{\hat{p}\}$ 
2: for  $t = 1$  to  $T$  do
3:    $\mathcal{C} \leftarrow \emptyset$ 
4:   for  $p \in \mathcal{B}_{t-1}$  do
5:     for  $j = 1$  to  $M$  do
6:        $p' \leftarrow e_j(p, t)$ 
7:       Execute  $p'$  with CadQuery
8:        $s(p') \leftarrow \text{IoU}(S(p'), m)$ 
9:        $\mathcal{C} \leftarrow \mathcal{C} \cup \{(p', s(p'))\}$ 
10:    end for
11:  end for
12:   $\mathcal{B}_t \leftarrow$  top- $K$  candidates in  $\mathcal{C}$  ranked by  $s(\cdot)$ 
13: end for
14: return  $\arg \max_{p \in \mathcal{B}_T} \text{IoU}(S(p), m)$ 
```

I Geometry-Aware Program Optimization Details

I.1 Beam Search Procedure

Given a seed program $\hat{p} = (a_1, \dots, a_T)$ sampled from the policy π_θ , we perform local optimization in program space:

$$p^* = \arg \max_{p \in \mathcal{N}(\hat{p})} \text{IoU}(S(p), m),$$

where $S(p)$ denotes the geometry produced by executing program p and $\mathcal{N}(\hat{p})$ is a neighborhood induced by local program edits.

Searching all edits jointly is exponential in the sequence length T . Instead, we exploit the sequential structure of CAD programs and perform a geometry-aware beam search over operations in execution order. At depth t , each candidate in the current beam is expanded by applying local edits to operation a_t , yielding:

$$\mathcal{B}_t = \text{TopK}_K \{p' \in \text{Expand}(p, t) : p \in \mathcal{B}_{t-1}\},$$

where candidates are ranked by:

$$\text{score}(p) = \text{IoU}(S(p), m).$$

We use beam width $K = 2$ and $M = 4$ local edits per expansion. At depth $t = 1$, the initial program expands into four candidates and the best two are retained. At subsequent depths, each surviving candidate produces four neighbors, resulting in eight evaluated candidates per depth.

The final refined program is selected as:

$$p^* = \arg \max_{p \in \mathcal{B}_T} \text{IoU}(S(p), m).$$

I.2 Continuous Parameter Refinement

The continuous refinement operator optimizes the scalar parameters of the current operation without modifying program structure. This includes extrusion depths, radii, sketch coordinates, offsets, and angles.

For an operation parameter vector

$$v \in \mathbb{R}^d,$$

we define a bounded local search region:

$$[v_i - r_i, v_i + r_i], \quad r_i = \min(\max(0.20|v_i|, 0.005), 0.30).$$

Optimization is performed using Bayesian Optimization with a Gaussian-Process surrogate and Expected-Improvement acquisition. We initialize the search with 6 random samples followed by 10 BO iterations using batches of 4 candidates each.

The refined parameters are accepted only if the resulting geometry improves IoU relative to the original operation.

I.3 Sketch Extraction and Replacement

The sketch modification operator replaces the sketch associated with the current operation using geometry extracted directly from the target mesh.

Given the operation working plane $(O, \vec{n}, \vec{x})$, we intersect the target mesh with the plane using `trimesh.section`. The resulting curves are projected into the local 2D coordinate system $(\vec{x}, \vec{y})$ and converted into closed polygons. The largest valid closed loop is selected.

The polygon is simplified using the Ramer–Douglas–Peucker algorithm, after which a primitive-fitting procedure selects among:

- full circles,
- line-and-arc decompositions,
- raw polygonal approximations.

The reconstructed sketch is then serialized back into CadQuery sketch commands:

```
Workplane(...).moveTo(...).lineTo(...)  
.threePointArc(...).close()
```

The original operation sketch is replaced while preserving the operation type and extrusion depth.

This edit is skipped if the plane does not intersect the target mesh or only open curves are produced.

I.4 Operation Skipping

The operation-skipping edit removes the current operation entirely:

$$p' = p \setminus a_t.$$

The modified program is executed and accepted only if:

$$\text{IoU}(S(p'), m) > \text{IoU}(S(p), m).$$

This edit is particularly effective for removing redundant operations, degenerate sketches, or erroneous boolean cuts introduced by the autoregressive policy.

I.5 Complement-Cut Optimization

The complement-cut operator removes overshoot regions by generating cut-extrudes from the boolean difference between the generated geometry and the target mesh.

Let:

$$G = S(p), \quad I = m.$$

We compute the overshoot region:

$$D = G \setminus I$$

using `manifold3d` boolean operations.

Planar faces of D are clustered by normal direction and plane offset using:

- angular tolerance: 2° ,
- offset tolerance: 10^{-3} .

Clusters with total area below 0.0025 are discarded.

For each planar cluster, boundary loops are projected into the local 2D plane and converted into sketches using the same primitive-fitting procedure as sketch replacement. Candidate cut-extrudes are generated by grid-searching extrusion depths:

$$\{0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.60, 0.80, 1.20, 2.00\}.$$

The best cut operation is appended to the program using:

```
combine='cut'.
```

We allow up to two complement-cut rounds, since removing one overshoot region can expose additional dominant errors.

Assets and licenses

We evaluate on CADBench, which aggregates data from publicly available datasets (e.g., ABC, Fusion 360, Objaverse). We follow the respective licenses and terms of use of the original sources. We also use pretrained models (Qwen2.5-Coder, Point-MAE) and CadQuery (Apache 2.0), in accordance with their licenses.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The claims in the abstract and introduction accurately reflect the papers contributions, methodology, and experimental results.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The limitations of the work are mentioned in section 5.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All the information needed to reproduce the main experimental results of the paper is available in the main paper and appendices.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We plan to release the code and dataset upon acceptance. The paper and supplementary material provide detailed implementation, training, and evaluation details intended to support reproducibility of the proposed approach and experiments.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The full details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not include statistical confidence intervals or error bars, primarily due to the high computational cost.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The full details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Our research complies with the NeurIPS Code of Ethics. The dataset is synthetic and poses no identifiable human data or privacy risks.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss societal impacts in Section 5.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All external datasets, models, and software used in this work are properly cited, and their corresponding licenses and terms of use are respected and referenced where applicable.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The paper and supplementary material document the proposed dataset, model, and evaluation pipeline, including details on generation, training, and limitations. We plan to release the assets and accompanying documentation upon acceptance.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The core method development in this research does not involve LLMs as any important, original, or non-standard components.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.